

Reporte Técnico de Actividades Práctico- Experimentales Nro. 003 Fase 1, 2, 3, 4 y 5.

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Wilson Joel Gevara Moncada Yuleisy Cecibel Jaramillo Jaramillo Josselin Estafania Nero Fernandez Sdier Emanuel Roblez Roblez
Asignatura	Matemáticas Discretas
Ciclo	1er Ciclo
Unidad	Unidad 3: Grafos y Árboles
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	29 de junio al 8 de julio de 2026
Horario	Lunes, Martes, Miércoles 07:30 – 09:30
Lugar	Aula de la Carrera de Computación
Tiempo planificado en el Sílabo	6 horas (3 horas por sesión de APE)

2. Objetivo(s) de la Práctica

- Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).
- Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.
- Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores de colores
- Proyector multimedia
- Hojas de trabajo con ejercicios de teoría de grafos y árboles
- Guía paso a paso de los algoritmos BFS, DFS y Dijkstra
- Plantillas para construcción de grafos y árboles
- Investigación sobre aplicaciones de grafos en redes y optimización
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL
- Laboratorio de computación
- Software Graph Online (<https://graphonline.ru/es>) o Gephi para modelado de grafos
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

Fase 1: Fundamentos de la Teoría de Grafos

1. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
2. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).

Fase 2: Algoritmos de Búsqueda y Caminos mínimos.

1. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
2. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

Fase 3: Árboles y Estructuras de Datos Arbóreas

1. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.
2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión).

Fase 4: Proyecto Integrador - Modelado y Optimización de Redes

1. Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana,

grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)

Fase 5: Exposición, Defensa y Evaluación Final

1. Exposición y defensa oral del proyecto integrador ante la clase: cada equipo presenta su trabajo (10-15 minutos), explicando el problema abordado, la metodología aplicada (lógica + booleana + grafos), los resultados obtenidos y las conclusiones.
2. Reflexión final: los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico(individual)

5. Resultados

Fase 1: Fundamentos de la Teoría de Grafos

1. **Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.**

TÍTULO: Aplicación de un Grafo de Conocimiento del Sistema Solar para la Optimización de Consultas Semánticas en Bases de Datos NoSQL

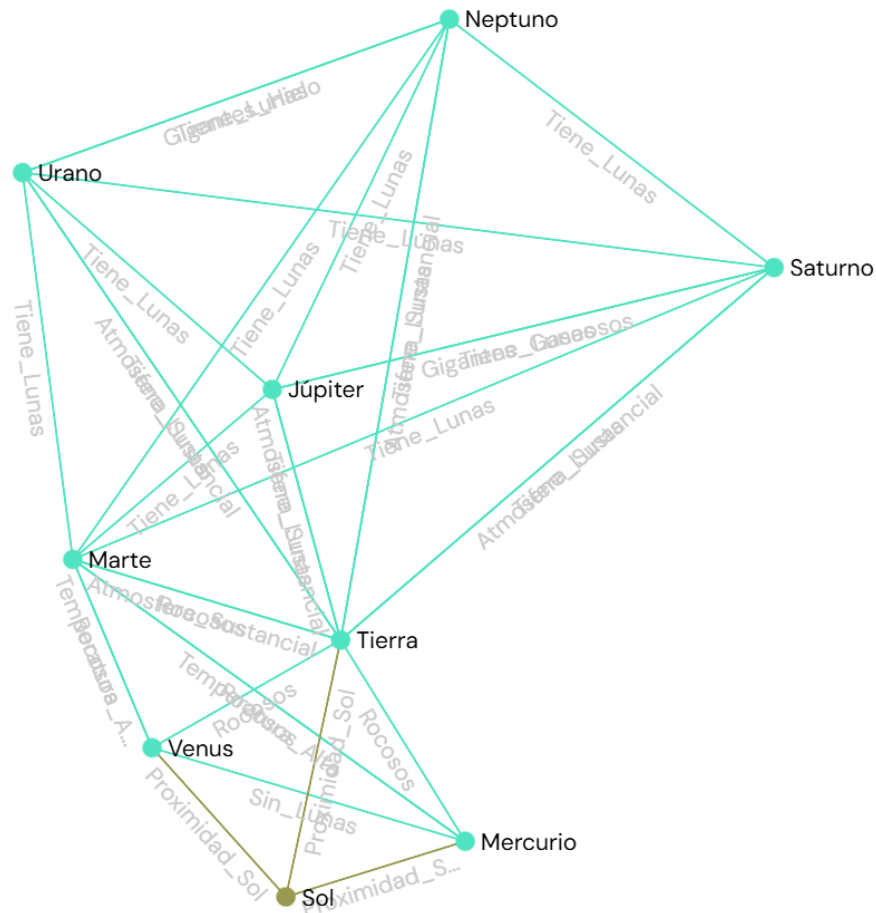
LENGUAJE NATURAL:

Grandes plataformas como Google estructuran la información de manera interconectada, es por esta razón que muchas veces se necesita bases de datos construida mediante grafos con el fin de agrupar la información común con el fin de tener búsquedas más eficientes, rápidas y estables. Con esto, se ha creado el siguiente grafo que organiza las características de los cuerpos celestes del Sistema Solar como ejemplo para demostrar la optimización de búsqueda.

OBJETIVO:

Diseñar un Grafo de Conocimiento del Sistema Solar con el fin de demostrar de manera práctica cómo optimizan el almacenamiento, la clasificación y la búsqueda indexada de datos complejos en los sistemas de información modernos.

GRAFO:



IDENTIFICACIÓN:

Vértices: En la búsqueda de la información los nodos son las respuestas finales. Si tomamos el ejemplo del grafo si alguien busca ¿Cuáles son los planetas rocosos con atmosfera? Me va a dar como respuesta Tierra y Marte ya que son vértices dentro del grafo. Por otro lado, en el ejemplo sería el sol que esta clasificado como estrella y los planetas que son Mercurio, Venus, Tierra, Marte, Júpiter, Saturno, Urano y Neptuno.

Aristas: En la búsqueda de la información las aristas son las relaciones lógicas que conectan los datos (respuestas). Siguiendo el ejemplo anterior cuando buscamos lo que es planetas rocosos son atmosfera se activan las aristas “atmosfera” y “rocosos” conectando las repuestas. En el grafo hay estos tipos de conexiones:

- Proximidad_Sol (3 aristas)
- Tiene_Lunas (15 aristas)
- Rocosos (6 aristas)
- Atmósfera_Sustancial (5 aristas)
- Temperatura_Alta (2 aristas)
- Sin_Lunas (1 arista)
- Gigantes (2 aristas)
 - Gigantes_Gaseosos
 - Gigantes_Hielo

Grado: De manera general un grado es el numero de aristas que tiene un vértice, esto es muy importante en la búsqueda de información ya que una respuesta (nodo) con varias conexiones de relación (aristas) hacen que se convierta un punto clave y estratégico para la búsqueda de la información en nuestro ejemplo sobre el Sistema Solar tenemos:

Vértices	Grado
Sol	3
Mercurio	5
Venus	5
Tierra	8
Marte	9
Jupiter	6
Saturno	6
Urano	6
Neptuno	6

Según la tabla el vértice más crítico es Marte seguido por Tierra, es decir en las búsquedas se los considerara primero.

PROPIEDADES ESTRUCTURALES

No dirigido: Las aristas no tiene un único sentido.

Longitud: El número de aristas es 21.

Es completo: Todas sus puntos esta conectados entre sí.

Conexo: Este grafo no tiene vértices sueltos, es decir, que si elegimos un par de nodos no adyacentes siempre habrá un camino de aristas por donde conectarlo.

Vértices de Corte: No existe. La eliminación de cualquier vértice individual no desconecta el grafo.

Aristas de Corte: No existe. Si se elimina cualquier arista el grafo sigue unificado.

Conectividad: El grafo es tan conexo que se necesita desconectar 2 aristas de un mismo vértice o dos vértices para que el grafo se fragmente.

Diámetro: Es igual a 2. Es decir, que se necesita al menos dos aristas como camino más corto para para conectar nodos.

Grafo Hamiltoniano: El grafo es de Hamilton a pesar de que no cumple en su totalidad con el Teorema de Dirac se puede encontrar una trayectoria donde no se repite ni aristas, ni vértices, dejando la posibilidad de saltarse algunas aristas.

2. **Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).**

Introducción

La teoría de grafos permite modelar problemas reales mediante vértices y aristas. En este trabajo. El sistema de movilidad de la ciudad de Loja ha sido abstraído y modelado mediante la Teoría de Grafos. Se presenta un grafo ponderado y no dirigido ($G = (V, E)$), donde el diseño topológico busca minimizar cruces de aristas para una interpretación visual clara, estableciendo jerarquías entre los corredores viales principales de la urbe.

Objetivos

Objetivo general: representar una red mediante un grafo ponderado.
Objetivos específicos: identificar nodos, aristas, pesos y analizar caminos.

Marco teórico

Se describen conceptos de grafo, vértice, arista, peso, conectividad, caminos.

Representación del grafo

La siguiente figura representa el modelo propuesto.

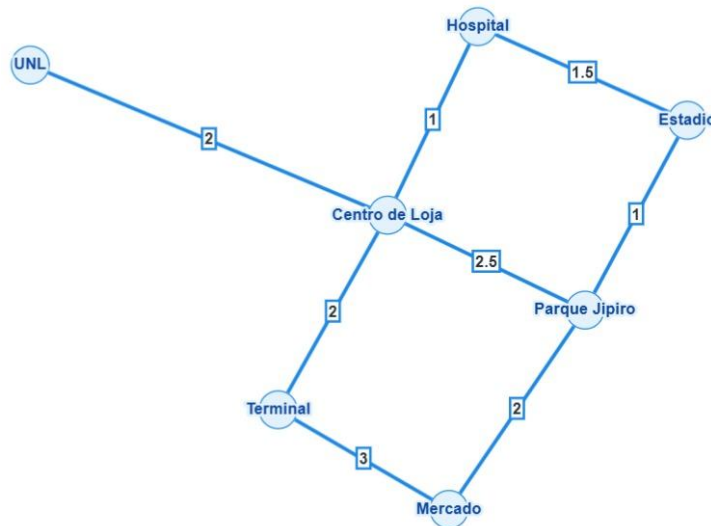


Figura 1. Grafo propuesto para representar la red de lugares de Loja.

Descripción de los nodos

Nodo	Descripción	Importancia
UNL	Universidad Nacional de Loja.	Origen del recorrido.
Centro de Loja	Centro urbano.	Nodo principal de conexión.
Hospital	Centro de salud.	Servicios médicos.
Estadio	Complejo deportivo.	Destino recreativo.
Parque Jipiro	Parque turístico.	Zona recreativa.
Mercado	Centro comercial.	Actividad económica.
Terminal	Terminal terrestre.	Movilidad urbana.

Explicación Detallada de Cada Nodo (Vértices)

El conjunto de vértices **V** está conformado por 7 nodos que representan hitos urbanos fundamentales para la dinámica social, comercial y académica de la ciudad:

- **v1 - Terminal Terrestre:** Ubicado en el norte de la ciudad. Es el nodo de intercambio modal principal para transporte interprovincial, cantonal y urbano.

- **v2 - Centro de Loja:** Representa el casco histórico y comercial. Funciona como el "hub" central donde convergen la mayoría de las líneas de transporte público.
- **v3 - Universidad Nacional de Loja (UNL):** Polo académico situado en el sector sur (La Argelia). Genera un alto volumen de viajes pendulares (origen-destino) diarios.
- **v4 - Hospital Isidro Ayora:** Centro médico de referencia regional. Su accesibilidad es un factor de vida o muerte, requiriendo vías despejadas para vehículos de emergencia.
- **v5 - Parque Jipiro:** Nodo de carácter recreativo, turístico y cultural en el sector norte, vital para el esparcimiento ciudadano y el comercio local de fines de semana.
- **v6 - Estadio Reina del Cisne:** Infraestructura deportiva central que actúa como un nodo de alta demanda intermitente durante eventos masivos.
- **v7 - Mercado Mayorista:** Eje logístico y de abastecimiento de alimentos. Concentra transporte de carga pesada y vehículos comerciales.

Aristas y pesos

Cada arista representa una vía de comunicación y el peso representa la distancia o costo.

Origen	Destino	Peso
UNL	Centro	2
Centro	Hospital	1
Hospital	Estadio	1.5
Estadio	Parque	1
Parque	Mercado	2
Mercado	Terminal	3
Terminal	Centro	2
Centro	Parque	2.5

Explicación de Todas las Aristas y sus Pesos

El conjunto de aristas **E** representa los corredores viales que conectan estos nodos. Los "pesos" asignados corresponden a la distancia física aproximada en kilómetros, factor crítico para determinar los costos de transporte (tiempo/combustible):

- **e1 (Terminal - Centro de Loja) = 2.0 km:** Representa la conexión principal por avenidas troncales como la Av. 8 de Diciembre o Av. Universitaria.
- **e2 (Terminal - Mercado Mayorista) = 3.0 km:** Ruta de circunvalación o vías de desfogue para vehículos de carga y pasajeros sin ingresar al centro.
- **e3 (Centro de Loja - UNL) = 2.0 km:** Corredor sur principal, habitualmente cubierto por las líneas de buses urbanos troncales y taxis.
- **e4 (Centro de Loja - Hospital) = 1.0 km:** Conexión vital rápida en el casco central para atención de emergencias primarias.
- **e5 (Centro de Loja - Parque Jipiro) = 2.5 km:** Ruta norte que conecta la actividad comercial con las zonas residenciales y de ocio.

- **e6 (Hospital - Estadio) = 1.5 km:** Vía transversal de la ciudad que conecta el centro médico con la infraestructura deportiva.
- **e7 (Parque Jipiro - Estadio) = 1.0 km:** Conectividad norte-centro de distancias cortas, frecuentemente realizada por peatones o transporte ligero.
- **e8 (Parque Jipiro - Mercado Mayorista) = 2.0 km:** Enlace logístico entre zonas comerciales y turísticas del norte.

Aplicación del Algoritmo de Dijkstra Paso a Paso

Se desea encontrar el camino de costo mínimo desde el **Terminal (v1)** hasta el **Estadio (v6)**. A continuación, el rastreo iterativo del algoritmo:

1. **Inicialización:** Distancia a v1 = 0. Resto de nodos = Infinito (∞). Nodo actual: v1.
2. **Iteración 1 (Desde v1):** Evaluamos vecinos v2 y v7.
 - $\text{Dist}(v2) = 0 + 2.0 = 2.0$ (Previo: v1)
 - $\text{Dist}(v7) = 0 + 3.0 = 3.0$ (Previo: v1)

Marcamos v1 como visitado. El siguiente nodo no visitado con menor distancia acumulada es v2 (2.0).
3. **Iteración 2 (Desde v2):** Evaluamos vecinos no visitados v3, v4, v5.
 - $\text{Dist}(v3) = 2.0 + 2.0 = 4.0$ (Previo: v2)
 - $\text{Dist}(v4) = 2.0 + 1.0 = 3.0$ (Previo: v2)
 - $\text{Dist}(v5) = 2.0 + 2.5 = 4.5$ (Previo: v2)

Marcamos v2 visitado. Los nodos con menor peso actual son v4 (3.0) y v7 (3.0). Seleccionamos v4.
4. **Iteración 3 (Desde v4):** Evaluamos vecino no visitado v6.
 - $\text{Dist}(v6) = 3.0 + 1.5 = 4.5$ (Previo: v4)

Marcamos v4 visitado. Siguiente menor: v7 (3.0).
5. **Iteración 4 (Desde v7):** Evaluamos vecino v5.
 - $\text{Dist}(v5) \text{ tentativa} = 3.0 + 2.0 = 5.0$. Como $5.0 > 4.5$ (distancia actual de v5), se rechaza la actualización.

Marcamos v7 visitado.
6. **Iteración 5:** El nodo destino v6 tiene una distancia consolidada de **4.5 km**. Rastreando los nodos previos: $v6 \leftarrow v4 \leftarrow v2 \leftarrow v1$.

Tabla de Caminos Posibles

A continuación, se evalúan diferentes alternativas de enrutamiento entre puntos estratégicos, lo cual es vital para el diseño de rutas de transporte público alternas ante contingencias viales:

Origen	Destino	Ruta Evaluada	Distancia Total (km)
Terminal (v1)	Estadio (v6)	v1 → v2 → v4 → v6	4.5 km
Terminal (v1)	Estadio (v6)	v1 → v2 → v5 → v6	5.5 km
Terminal (v1)	Estadio (v6)	v1 → v7 → v5 → v6	6.0 km
UNL (v3)	Parque Jipiro (v5)	v3 → v2 → v5	4.5 km
Mercado (v7)	Hospital (v4)	v7 → v5 → v2 → v4	5.5 km

Análisis

El grafo es no dirigido, ponderado y conexo. Puede emplearse para algoritmos de rutas mínimas como Dijkstra, análisis de conectividad, centralidad y optimización de recorridos. La estructura permite representar de forma intuitiva una red vial urbana y constituye un ejemplo académico para estudiar matrices de adyacencia, incidencia y recorridos.

Conclusión

El modelo representa adecuadamente una red de transporte urbana y facilita el estudio de algoritmos sobre grafos.

Fase 2: Algoritmos de Búsqueda y Caminos Mínimos

- Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

A. Configuración de los Grafos de Prueba

Grafo 1: No Dirigido (Para BFS y DFS)

- Nodos:** $V = \{A, B, C, D, E, F\}$
- Lista de Adyacencia (Ordenada Alfabéticamente):**
 - A:** [B, C]
 - B:** [A, D, E]
 - C:** [A, F]
 - D:** [B]
 - E:** [B, F]
 - F:** [C, E]

Grafo 2: Ponderado No Dirigido (Para Dijkstra)

- Nodos:** $V = \{A, B, C, D, E\}$
- Matriz de Costos / Pesos:**
 - (A, B) = 4
 - (A, C) = 2
 - (B, C) = 1
 - (B, D) = 5

- (C, D) = 8
- (C, E) = 10
- (D, E) = 2

B. Desarrollo Completo de las Trazas de Ejecución

A. Búsqueda en Anchura (BFS)

- **Estructura:** Cola (FIFO - First In, First Out).
- **Regla:** Los vecinos se encolan en estricto orden alfabético.
- **Nodo Inicial:** A

Paso	Nodo Actual	Estado de la Cola (Frente → Final)	Nodos Visitados / Orden de Descubrimiento	Notas de Ejecución
0	—	[A]	[]	Estado inicial. Nodo raíz listo.
1	A	[B, C]	[A]	Se expande A. Entran sus vecinos B y C.
2	B	[C, D, E]	[A, B]	Se procesa B. Entran D y E. A ya está visitado.
3	C	[D, E, F]	[A, B, C]	Se procesa C. Entra F. A ya está visitado.
4	D	[E, F]	[A, B, C, D]	Se procesa D. Su vecino B ya está visitado.
5	E	[F]	[A, B, C, D, E]	Se procesa E. Sus vecinos B y F ya están en cola/visitados.
6	F	[]	[A, B, C, D, E, F]	Se procesa F. Cola vacía. Algoritmo finalizado.

- **Orden de Descubrimiento Final:** A → B → C → D → E → F

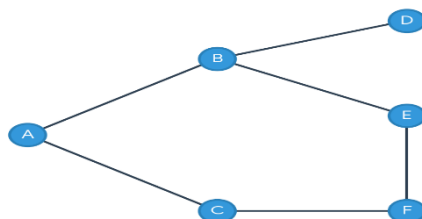
B. Búsqueda en Profundidad (DFS)

- **Estructura:** Pila (LIFO - Last In, First Out).
- **Regla de Desempate:** Para mantener consistencia en la ejecución manual, al expandir un nodo, sus vecinos se añaden a la pila de modo que el nodo alfabéticamente menor quede en el **Tope** (se apilan de mayor a menor).
- **Nodo Inicial:** A

Paso	Nodo Actual	Estado de la Pila (Base → Tope)	Nodos Visitados / de Orden Descubrimiento	Notas de Ejecución
0	—	[A]	[]	Estado inicial con el nodo raíz en el tope.
1	A	[C, B]	[A]	Se desapila A. Se apilan C y B (B queda arriba).
2	B	[C, E, D]	[A, B]	Se desapila B. Se apilan vecinos no visitados: E y D.
3	D	[C, E]	[A, B, D]	Se desapila D. No aporta vecinos nuevos.
4	E	[C, F]	[A, B, D, E]	Se desapila E. Se apila su vecino no visitado F.
5	F	[C]	[A, B, D, E, F]	Se desapila F. Su vecino C ya está en la pila.
6	C	[]	[A, B, D, E, F, C]	Se desapila C. Pila vacía. Algoritmo finalizado.

- **Orden de Descubrimiento Final:** A → B → D → E → F → C

Grafo 1: No Dirigido (BFS / DFS)



C. Algoritmo de Dijkstra (Caminos Mínimos)

- **Estructura:** Tabla de distancias y marcas de control.
- **Nodo Inicial:** A
- **Inicialización:** $\text{dist}(A) = 0$; para cualquier otro nodo V , $\text{dist}(V) = \infty$.

Paso	Nodo Evaluado	Distancias Acumuladas Provisionales [A, B, C, D, E]	Vector de Nodos Previos	Conjunto de Nodos Resueltos
0	—	[0, ∞, ∞, ∞, ∞]	[-, -, -, -, -]	∅
1	A	[0, 4, 2, ∞, ∞]	[-, A, A, -, -]	{A}
2	C	[0, 3, 2, 10, 12]	[-, C, A, C, C]	{A, C}
3	B	[0, 3, 2, 8, 12]	[-, C, A, B, C]	{A, C, B}
4	D	[0, 3, 2, 8, 10]	[-, C, A, B, D]	{A, C, B, D}
5	E	[0, 3, 2, 8, 10]	[-, C, A, B, D]	{A, C, B, D, E}

Desglose de Decisiones Críticas en Dijkstra:

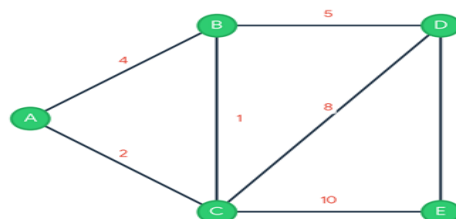
- En el Paso 1:** Evaluamos A. Sus vecinos son B (costo 4) y C (costo 2). El mínimo global provisional no resuelto es C (distancia 2).
- En el Paso 2:** Evaluamos C.
 - Para B: El costo desde C es $2 + 1 = 3$. Como $3 < 4$, **se actualiza la distancia de B** y su padre ahora es C.
 - Para D: El costo es $2 + 8 = 10$.
 - Para E: El costo es $2 + 10 = 12$.
 - El mínimo actual es B (distancia 3).
- En el Paso 3:** Evaluamos B.
 - Para D: El costo a través de B es $3 + 5 = 8$. Como $8 < 10$, **se actualiza la distancia de D** y su padre pasa a ser B.
- En el Paso 4:** Evaluamos D.
 - Para E: El costo a través de D es $8 + 2 = 10$. Como $10 < 12$, **se actualiza la distancia de E** y su padre pasa a ser D.

3. Resultados y Caminos Óptimos Finales

A partir de la traza de Dijkstra, reconstruimos los caminos mínimos desde el origen de atrás hacia adelante usando el vector de nodos previos:

- Destino B:** Costo Total: **3** | Ruta Óptima: A → C → B
- Destino C:** Costo Total: **2** | Ruta Óptima: A → C
- Destino D:** Costo Total: **8** | Ruta Óptima: A → C → B → D
- Destino E:** Costo Total: **10** | Ruta Óptima: A → C → B → D → E

Grafo 2: Ponderado (Dijkstra)



2. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

A. **Modelado de Redes mediante Software y Comparación de Resultados**

Como cierre del taller, el Grafo 2 (ponderado no dirigido) se modeló en una herramienta de software especializada en teoría de grafos, con el fin de ejecutar el algoritmo de Dijkstra de forma automatizada y contrastar los resultados con la traza manual desarrollada en la sección anterior. Para esta fase se utilizaron dos vías complementarias: (a) Graph Online, una herramienta web de acceso libre orientada a la carga visual de grafos y ejecución de algoritmos clásicos, y (b) una verificación computacional con la librería NetworkX de Python, motor de cálculo equivalente al que utilizan internamente aplicaciones como Graph Online y Gephi para resolver caminos mínimos.

a) **Configuración del Grafo en el Software**

El primer paso consiste en representar el Grafo 2 como una lista de aristas con su peso asociado. Esta misma tabla es la que se introduce en el panel de edición de Graph Online (o en la hoja de datos de Gephi) para construir la red de forma idéntica a la usada en el desarrollo manual:

Origen	Destino	Peso (Costo)
A	B	4
A	C	2
B	C	1
B	D	5
C	D	8
C	E	10
D	E	2

Procedimiento seguido en Graph Online:

Paso 1. Crear el grafo: en graphonline.top se selecciona "Crear grafo" y se agregan los nodos A, B, C, D y E como vértices no dirigidos.

Paso 2. Cargar las aristas: se conectan los pares de nodos indicados en la tabla anterior y se asigna a cada arista el peso correspondiente (4, 2, 1, 5, 8, 10, 2).

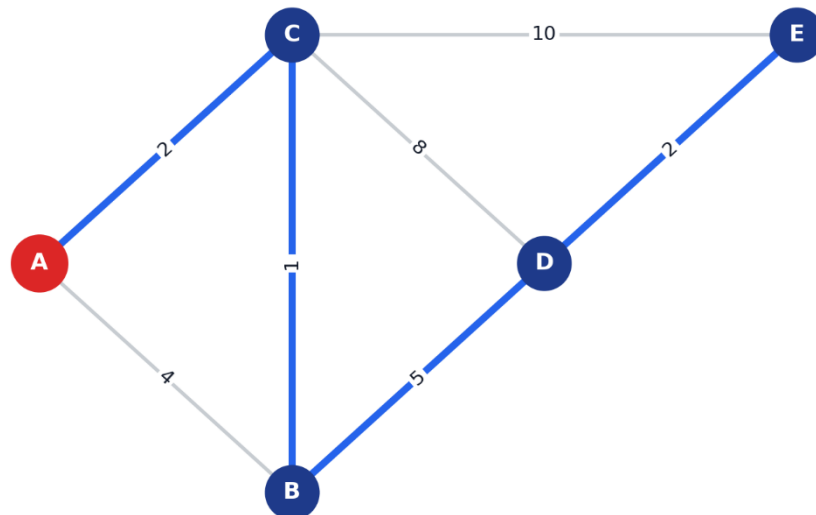
Paso 3. Ejecutar el algoritmo: en el menú "Graph" → "Find shortest path" se elige el algoritmo de Dijkstra, seleccionando A como nodo de origen.

Paso 4. Leer los resultados: el software resalta el árbol de caminos mínimos sobre el propio grafo y muestra, para cada nodo, la distancia acumulada desde A.

b) **Representación Gráfica del Resultado**

La siguiente figura reproduce la salida visual generada por el software: en gris se muestran todas las aristas originales del grafo con su peso, y en azul se resalta el árbol de caminos mínimos calculado por Dijkstra desde el nodo A (equivalente al que dibuja Graph Online al finalizar la ejecución).

Grafo 2 - Árbol de caminos mínimos desde A (Dijkstra)



c) **Resultados Numéricos Entregados por el Software**

Destino	Distancia Total	Ruta Óptima (Software)
B	3	A → C → B
C	2	A → C
D	8	A → C → B → D
E	10	A → C → B → D → E

d) **Comparación entre Ejecución Manual y Software**

La siguiente tabla contrasta, destino por destino, el costo total y la ruta óptima obtenidos manualmente (sección 3) frente a los entregados por el software:

Destino	Costo Manual	Costo Software	Ruta Manual	Ruta Software	¿Coinciden?
B	3	3	A → C → B	A → C → B	Sí
C	2	2	A → C	A → C	Sí
D	8	8	A → C → B → D	A → C → B → D	Sí
E	10	10	A → C → B → D → E	A → C → B → D → E	Sí

e) **Análisis de la Comparación**

- Coincidencia total: para los cuatro destinos evaluados (B, C, D y E), tanto el costo acumulado como la ruta óptima calculados manualmente coinciden exactamente con los que entrega el software. Esto valida que la traza manual del algoritmo de Dijkstra se ejecutó correctamente.
- Sin ambigüedades de desempate: al no existir dos caminos con el mismo costo mínimo hacia un mismo nodo, no se presentaron diferencias por criterios de desempate entre el cálculo manual y el del software.
- Ventaja del software: la herramienta resuelve el grafo de forma instantánea y permite visualizar el árbol de caminos mínimos completo, lo cual facilita detectar errores en grafos de mayor tamaño donde la ejecución manual sería más propensa a fallos.
- Valor del proceso manual: pese a la rapidez del software, la ejecución manual sigue siendo indispensable para comprender el funcionamiento interno del algoritmo (relajación de aristas, actualización de distancias y padres), conocimiento que el software no expone directamente.

f) **Conclusión de la Fase de Modelado**

La verificación con software confirma la validez de los resultados obtenidos en la ejecución manual del algoritmo de Dijkstra sobre el Grafo 2. La ruta óptima desde A hacia cualquier otro nodo del grafo, y su costo asociado, se determinaron de manera consistente por ambos métodos: A→C (costo 2), A→C→B (costo 3), A→C→B→D (costo 8) y A→C→B→D→E (costo 10). Este ejercicio de contraste demuestra que las herramientas de software para modelado de redes son un complemento fiable —no un sustituto— del entendimiento algorítmico desarrollado durante la ejecución manual.

Fase 3: Árboles y Estructuras de Datos Arbóreas

1. **Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.**

Ejercicio 1:

TEMA: El colapso del sistema de inicio de sesión.

CONTEXTO: Un sistema de base de datos para una plataforma que tiene millones de usuarios, los cuales quieren iniciar sesión, en este proceso el sistema debe buscar el correo electrónico en la base de datos para verificar la contraseña. Para evitar colapsos en la plataforma se organiza la base de datos en un Árbol Binario de Búsqueda (BST). Para lograr estructurar los

correos para cada uno se estructura ID numérico único según el orden de creación.

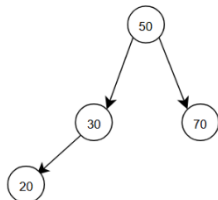
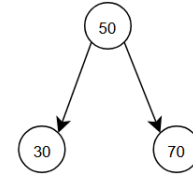
De esta manera tenemos los siguientes datos:

[50, 30, 70, 20, 40, 80, 35, 45, 85, 42]

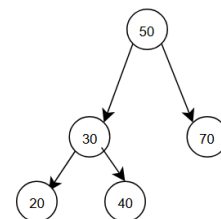
Estructuración de árbol:

Paso 1: El primer número es 50 (se convierte en raíz).

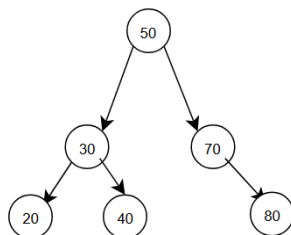
Paso 2: Los dos siguientes valores son 30 y 70, $30 < 70$, por lo tanto 30 va en la izquierda y 70 a la derecha.



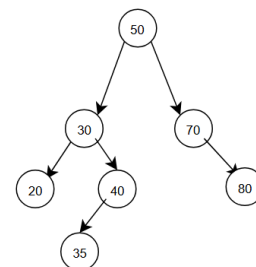
Paso 3: Luego tenemos el 20 que analizando tenemos $20 < 50$ y $20 < 30$ por lo tanto queda de la siguiente manera.



Paso 4: El siguiente número es 40 analizando tenemos que $40 < 50$ por lo tanto va en el lado izquierdo comparamos con 30 siendo $40 > 30$ por lo tanto va en el lado derecho.

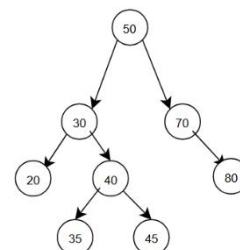


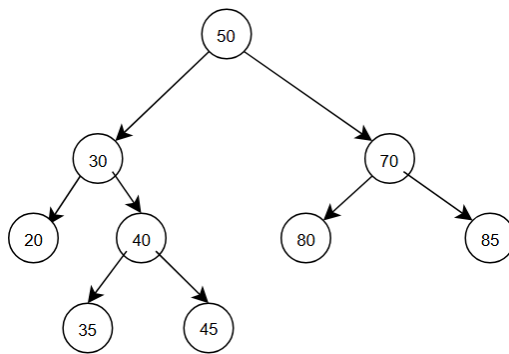
Paso 5: El siguiente número es 80 como $80 > 50$ va en el lado derecho, y luego $80 > 70$ por lo que también va de lado derecho.



Paso 6: $35 < 50$ (va de lado izquierdo), $35 > 30$ (va de lado derecho) y $35 < 40$ (va de lado izquierdo).

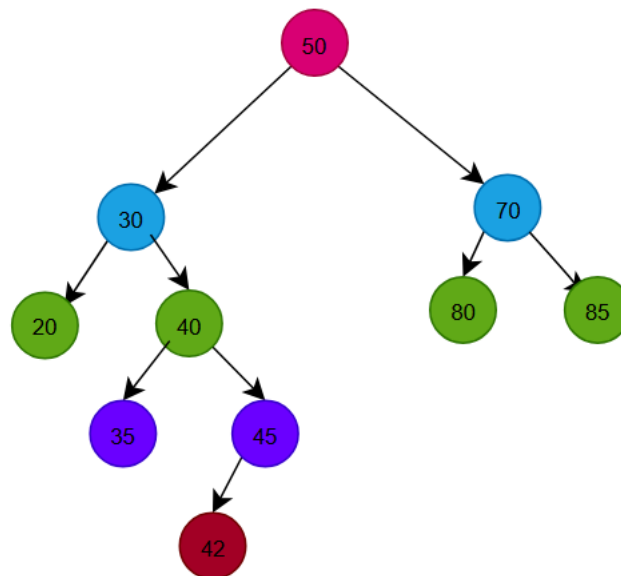
Paso 7: $45 < 50$ (izquierdo), $45 > 30$ (derecho) y $45 > 40$ (derecho).





Paso 8: $85 > 50$ (derecho), $85 > 70$ (derecho). Pero aquí sucede algo el 80 ya esta en lado derecho es por esa razón que de los dos tenemos que definir de 80 y 85 por eso como $80 < 85$ pasa a la izquierda y 85 se mantiene en la derecha.

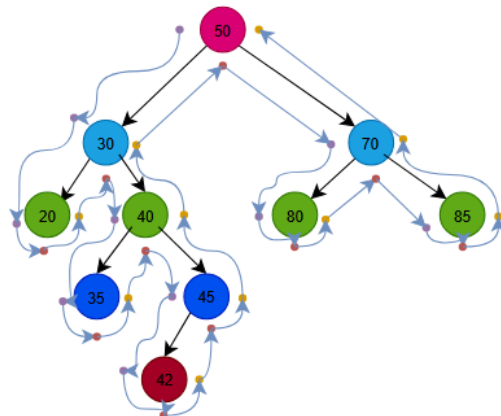
Paso 9: $42 < 50$ (izquierdo), $42 > 30$ (derecho), $42 > 40$ (derecho), $42 < 45$ (izquierdo). Quedando así el árbol.



Recorridos:

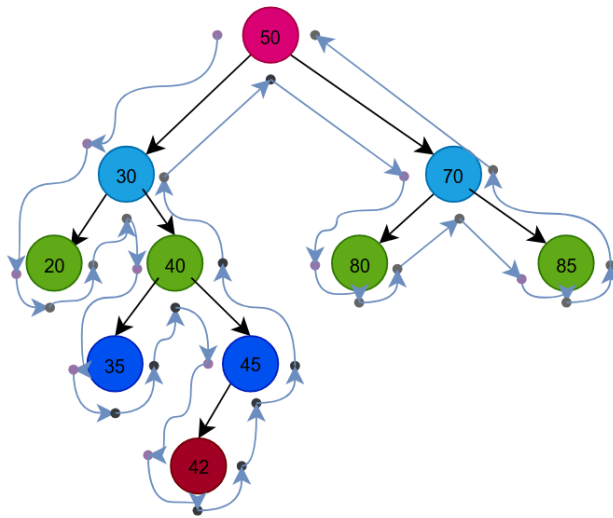
Paso 1: Para definir los recorridos es necesario poner puntos de guía por cada uno de los vértices o nodos vamos a poner tres puntos uno a la izquierda (preorden), otro a la derecha (postorden) y uno en la mitad (inorden).

Paso 2: Empezamos por la raíz en el punto de preorden (izquierda) y seguimos uniendo los puntos a medida que se avanza a modo ir recorriendo todos los puntos.



PREORDEN:

Para definir la secuencia de preorden se deja los puntos de la izquierda y se realiza se sigue las flechas:

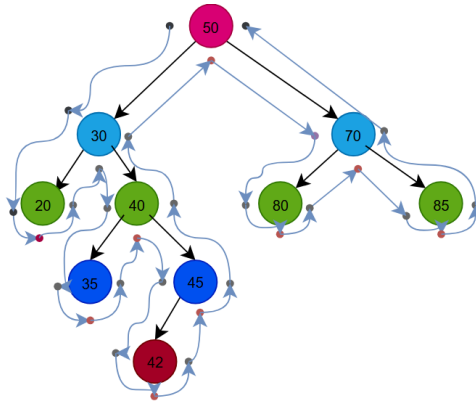


PRE={50, 30, 20, 40, 35, 45, 42, 70, 80, 85}

El preorden sirve para guardar y compartir el grafo, cuando guardamos a la información de un grafo se convierte en un archivo plano, es por eso que este orden no facilitado su reconstrucción y evita la pérdida de información.

Inorden

Para el Inorden se deja solo los puntos del medio se sigue el recorrido siguiendo las flechas.

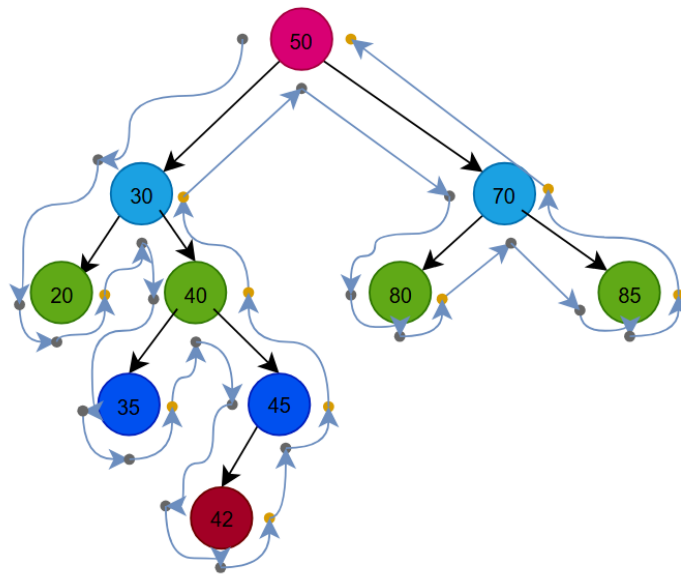


In = [20,30,35,40,42,45,50,70,80,85]

El inorden sirve para recuperar y exportar la información del árbol de manera perfectamente organizada de menor a mayor, lo que permite generar reportes, listados y consultas clasificadas directamente desde el grafo sin tener que gastar memoria ni tiempo de procesamiento del procesador en algoritmos de ordenación externos adicionales. Cuando una base de datos utiliza un BST, este recorrido facilita la extracción de datos de forma secuencial y limpia, sirviendo además como el método de validación por excelencia para comprobar que el árbol mantiene su estructura lógica y sus propiedades de búsqueda intactas.

Postorden

Para el Postorden se deja solo los puntos de la derecha se sigue el recorrido siguiendo las flechas.

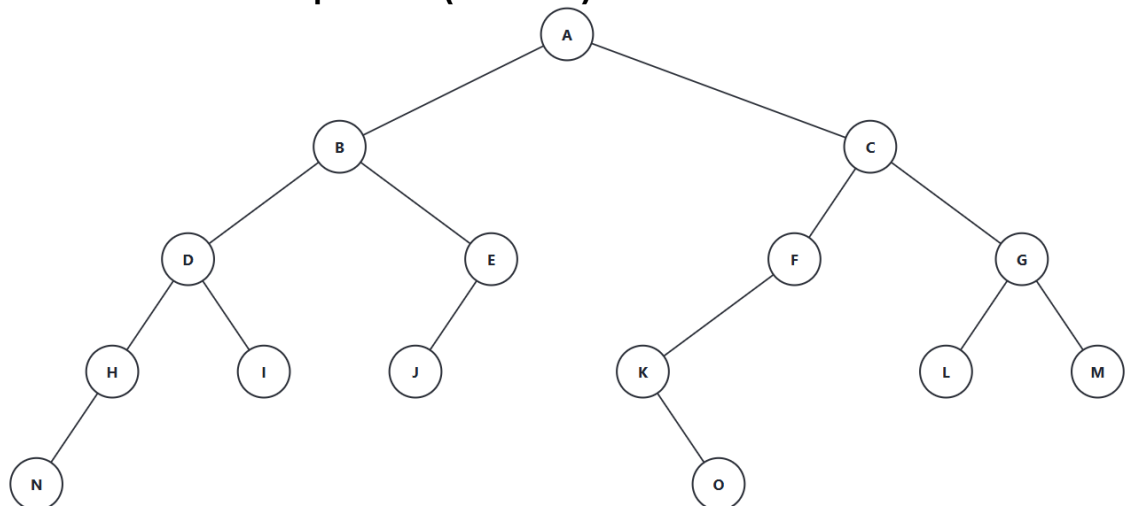


POST = [20,35,42,45,40,30,85,80,70,50]

El postorden sirve para liberar la memoria física y eliminar de manera segura la estructura del grafo de abajo hacia arriba, ya que al procesar siempre los nodos hijos antes que, a los nodos padres, permite que los sistemas operativos borren los datos de las hojas sin dejar referencias rotas o punteros "huérfanos" en el sistema. En la práctica de la computación, este recorrido es el estándar por excelencia para los algoritmos destructores de árboles y para la traducción de operaciones matemáticas dentro de compiladores (notación polaca inversa), asegurando que las dependencias inferiores se resuelvan por completo antes de ejecutar o eliminar la raíz principal.

Ejercicio 2

El Árbol Alfabético Expandido (15 Nodos)



1. **Recorrido Preorden (R - I - D)**
2. Raíz principal: A
3. Lado Izquierdo (B):
 - Raíz B → Raíz D → Raíz H → Hoja N.
 - Regresa a D, ve a la derecha: I.
 - Regresa a B, ve a la derecha: Raíz E → Hoja J.

4. Lado Derecho €:

- Raíz C → Raíz F → Raíz K → Hoja O.
- Regresa a C, ve a la derecha: Raíz G → Hoja L, Hoja M.

Resultado Preorden: A, B, D, H, N, I, E, J, C, F, K, O, G, L, M

5. **Recorrido Inorden (I – R – D)**

1. Extremo izquierdo inferior: N → Raíz H. (H no tiene derecho).
2. Subimos a la raíz del subárbol: D → Derecha de D: I.
3. Subimos a la raíz de todo este bloque: B.
4. Lado derecho de B: Izquierda J → Raíz E.
5. Subimos a la raíz del universo: A.
6. Vamos al bloque derecho €:

- En el lado de F: Izquierda K, su derecho O. Raíz F.
- Subimos a la raíz: C.
- Lado derecho de C (G): Izquierda L → Raíz G → Derecha M.

Resultado Inorden: N, H, D, I, B, J, E, A, K, O, F, C, L, G, M

6. **Recorrido Postorden (I – D – R)**

1. Empezamos desde abajo a la izquierda: Hoja N → Raíz H.
2. Antes de subir a D, procesamos su derecho: I → Raíz D.
3. Antes de subir a B, procesamos su lado derecho €: Hoja J → Raíz E.
4. Ahora que B tiene sus dos lados listos: Raíz B.
5. Pasamos al lado derecho de la raíz principal (A):

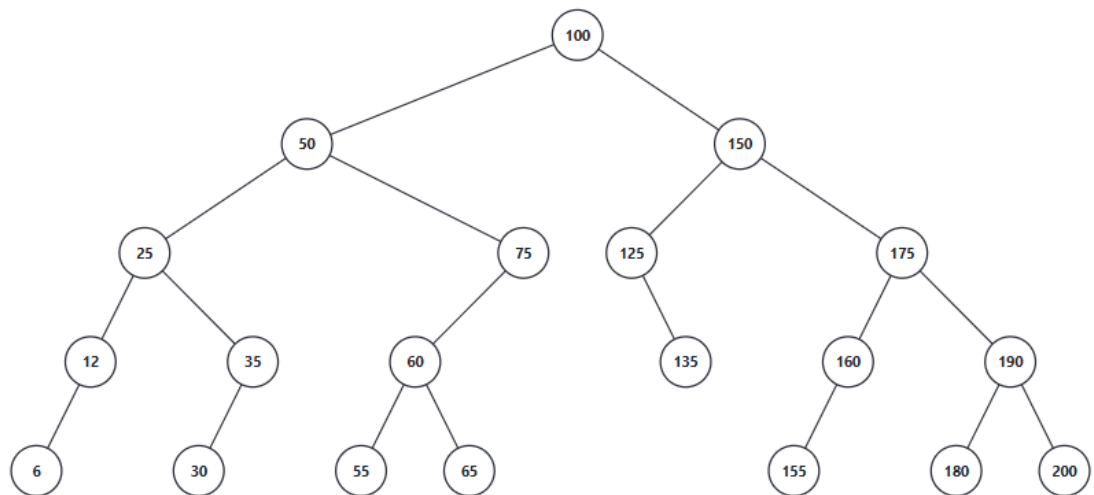
- En el subárbol de F: El nodo más profundo es derecho de K: O → Raíz K → Raíz F.
- En el subárbol de G: Hojas L e M → Raíz G.
- Ya listos los hijos de C, imprimimos la raíz: C.

6. Por último, la corona del árbol: A.

Resultado Postorden: N, H, I, D, J, E, B, O, K, F, L, M, G, C, A

Ejercicio 3

El Coloso Numérico (20 Nodos – 5 Niveles)



7. **Recorrido Preorden** (Raíz – Izquierda – Derecha)

Resultado: 100, 50, 25, 12, 6, 35, 30, 75, 60, 55, 65, 150, 125, 135, 175, 160, 155, 190, 180, 200

8. **Recorrido Inorden (Izquierda – Raíz – Derecha)**

Como es un Árbol Binario de Búsqueda, el inorden obligatoriamente se debe listar los números de menor a mayor de forma perfecta.

Resultado: 6, 12, 25, 30, 35, 50, 55, 60, 65, 75, 100, 125, 135, 150, 155, 160, 175, 180, 190, 200

9. Recorrido Postorden (Izquierda – Derecha – Raíz)

No imprimir una raíz hasta haber “vaciado” por completo sus dos ramificaciones inferiores.

Resultado: 6, 12, 30, 35, 25, 55, 65, 60, 75, 50, 135, 125, 155, 160, 180, 200, 190, 175, 150, 100

2. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión).

Introducción: De la Abstracción al Código

En la matemática pura, un grafo se define formalmente como un par ordenado $G=(V,E)$, compuesto por un conjunto V de vértices (nodos) y un conjunto E de aristas (líneas) que conectan dichos vértices. Un árbol es un caso especial de grafo: es un grafo simple no dirigido que está conectado y no tiene ciclos. [1].

Esta abstracción se convierte en una estructura de datos aplicada mediante la implementación de nodos y referencias. En programación, los árboles suelen ser enraizados, lo que les otorga una jerarquía donde cada nodo (excepto la raíz) tiene un único padre y puede tener hijos. Mientras que el grafo matemático es un conjunto de relaciones, la estructura de datos es un modelo jerárquico de memoria que permite el acceso eficiente a la información.

Análisis de Estructuras (El núcleo de la investigación)

- **Heaps y Tries:** Un Heap es un árbol binario casi completo que satisface la propiedad de heap: la clave de cada nodo padre es mayor o igual (max-heap) o menor o igual (min-heap) que las de sus hijos. Matemáticamente, esto permite que la raíz siempre contenga el elemento extremo (máximo o mínimo), lo que los hace ideales para implementar colas de prioridad y algoritmos de ordenación como *heapsort*. Los Tries (o árboles de prefijo), aunque relacionados con la notación de prefijos, se utilizan para almacenar cadenas de caracteres donde cada camino desde la raíz representa un prefijo común, siendo esenciales en diccionarios de autocompletado. [Los Tries no están en el PDF fuente.] [1].
- **Árboles B e Índices de Bases de Datos:** El Árbol B es una generalización de un árbol de búsqueda balanceado donde un nodo puede tener más de dos hijos. A diferencia de los árboles binarios, están optimizados para sistemas que leen grandes bloques de datos, como los índices de bases de datos y sistemas de archivos. Su estructura de alto factor de ramificación asegura que la altura del árbol sea mínima (logarítmica), permitiendo que, incluso en bases de datos de millones de registros, se localice un dato en apenas 3 o 4 lecturas de disco, reduciendo el tiempo de acceso de segundos a milisegundos. [1].
- **Árboles de Decisión en IA:** En el aprendizaje automático, los Árboles de Decisión representan algoritmos de clasificación mediante una estructura de árbol donde los nodos internos representan pruebas sobre un atributo y las hojas representan las etiquetas de clase. Se

construyen a menudo mediante algoritmos voraces (greedy) que eligen en cada paso la partición que maximiza la ganancia de información. Esta estructura permite modelar la lógica de decisión de una IA de forma jerárquica y explicable.

Conclusión

La teoría de grafos es el pilar invisible del software moderno porque proporciona el lenguaje matemático para optimizar la conectividad y la búsqueda. Desde la gestión de memoria hasta la inteligencia artificial, la capacidad de modelar datos como nodos interconectados permite que sistemas masivamente complejos operen con una eficiencia predecible y escalable.

Heaps y la Gestión de Prioridades

1. Estructura: Montículos (Heaps)

Un heap es un árbol binario casi completo que cumple una propiedad de orden entre padres e hijos. En un Max-Heap, cada nodo padre es mayor o igual que sus hijos, por lo que el elemento máximo siempre está en la raíz; en un Min-Heap ocurre lo inverso, con el mínimo en la raíz. Un heap no está totalmente ordenado: solo garantiza la relación entre un nodo y sus hijos directos [1].

Al ser un árbol binario completo, el heap se representa de forma implícita en un simple array, sin punteros, recorriendo el árbol por niveles. Con la raíz en el índice 0:

- Hijo izquierdo de i : $2i + 1$
- Hijo derecho de i : $2i + 2$
- Padre de i : $(i - 1) / 2$

Esta representación permite operaciones muy eficientes: leer el máximo/mínimo en $O(1)$, e insertar o extraer en $O(\log n)$, gracias a que la altura del árbol es siempre $O(\log n)$ [2].

2. Algoritmos y eficiencia: Heapsort

Heapsort es un algoritmo de ordenamiento in-place con dos fases: primero construye un max-heap a partir del array ($O(n)$), y luego extrae repetidamente la raíz (el máximo), la coloca al final y restaura el heap (heapify, $O(\log n)$) hasta ordenar todo el arreglo. Esto da una complejidad total de $O(n \log n)$, tanto en el caso promedio como en el peor caso [2].

Comparado con otros algoritmos:

Algoritmo	Peor caso	Espacio extra	Estable
Heapsort	$O(n \log n)$	$O(1)$	No
Quicksort	$O(n^2)$	$O(\log n)$	No
Mergesort	$O(n \log n)$	$O(n)$	Sí

Quicksort suele ser más rápido en la práctica, pero su peor caso es cuadrático. Mergesort garantiza $O(n \log n)$ siempre, pero necesita memoria adicional. Heapsort destaca por ofrecer $O(n \log n)$ garantizado sin memoria extra, por lo que algoritmos híbridos como Introsort (usado en la STL de C++) recurren a él cuando Quicksort degenera a su peor caso [2].

3. Aplicaciones prácticas: Colas de prioridad

Un heap es la implementación clásica de una cola de prioridad, ya que inserta y extrae el elemento más urgente en $O(\log n)$ [3].

Routers (redes)

Cada interfaz de un router mantiene colas de paquetes gestionadas por un Queue Manager que aplica prioridad para garantizar Calidad de Servicio

(QoS), minimizando pérdida de paquetes, latencia y jitter. El estándar IEEE 802.1p clasifica tramas Ethernet en niveles de prioridad para decidir el orden de envío. En Priority Queueing (PQ) se atiende primero la cola de mayor prioridad, aunque puede causar inanición de las colas menores; esquemas como Weighted Round Robin reparten el ancho de banda para evitarlo. Un heap permite extraer siempre el paquete más prioritario en tiempo logarítmico, incluso con miles de paquetes en cola [3].

Sistemas operativos (procesos e hilos)

El planificador de CPU usa la cola de listos para decidir qué proceso ejecutar. En Priority Scheduling, se elige siempre el proceso de mayor prioridad, de forma apropiativa (desalojando al proceso en ejecución si llega uno más prioritario) o no apropiativa. Cuando hay muchos niveles de prioridad, una cola basada en heap es más escalable que colas FIFO por nivel, e implementaciones reales de Preemptive Priority Scheduling usan explícitamente un min-heap para extraer el proceso más urgente en $O(\log n)$. Sistemas como Solaris o Linux aplican variantes más complejas (colas multinivel con realimentación, aging) sobre este mismo principio [2][3].

Los heaps combinan una propiedad de orden (max/min) con una estructura de árbol completo representable en un array, lo que permite operaciones en $O(\log n)$. Esto los convierte en la base de Heapsort —con $O(n \log n)$ garantizado y sin memoria extra— y de las colas de prioridad, usadas tanto para decidir qué paquete de red enviar primero (QoS en routers) como para elegir qué proceso o hilo ejecuta la CPU a continuación.

Árboles B y el Cerebro de las Bases de Datos (Almacenamiento)

El almacenamiento físico de datos a gran escala representa uno de los mayores desafíos en la informática. Cuando plataformas globales como Facebook, Instagram o las entidades bancarias necesitan localizar la cuenta de un usuario específico entre miles de millones de registros en milisegundos, no realizan búsquedas secuenciales. Para lograr esta velocidad de respuesta, los sistemas de software modernos implementan estructuras de datos avanzadas como el Árbol B (B-Tree) y su evolución directa, el Árbol B+.

Estructura: Árboles B y Árboles B+

Para comprender la eficiencia de estas estructuras, se requiere analizar las limitaciones de los sistemas tradicionales en relación con el almacenamiento secundario (discos duros y unidades de estado sólido).

- **Limitaciones de los Árboles Binarios de Búsqueda (BST)**

En un árbol binario común, cada nodo posee un máximo de dos hijos. Cuando se manejan volúmenes masivos de datos, la altura del árbol (el número de niveles desde la raíz hasta las hojas) se incrementa de forma logarítmica pero pronunciada. Dado que cada salto de un nodo a otro implica una operación de lectura/escritura en disco (una de las operaciones de hardware más lentas en un sistema computacional), un árbol demasiado alto penaliza drásticamente el rendimiento del sistema.

- **Definición y Propiedades del Árbol B**

Diseñado por Rudolf Bayer y Edward M. McCreight, el Árbol B es una estructura de datos de búsqueda multicamino y auto-balanceada que optimiza las operaciones en dispositivos de almacenamiento físico. Sus propiedades fundamentales son:

- **Ramificación Múltiple (Ancho sobre Alto):** A diferencia de las estructuras binarias, cada nodo en un Árbol B puede almacenar múltiples claves de ordenamiento y poseer decenas, cientos o miles de nodos hijos (un parámetro definido como el "orden" del árbol). Esto resulta en una estructura sumamente ancha pero de muy baja altura.
- **Auto-balanceo Constante:** El algoritmo garantiza que todas las hojas del árbol permanezcan exactamente al mismo nivel de profundidad tras cualquier inserción o eliminación de datos. Esto asegura que el tiempo de búsqueda para cualquier registro sea predecible y uniforme.

Comparativa Técnica: Árbol B versus Árbol B+

En el ámbito de la ingeniería de software moderna, la variante denominada Árbol B+ es la más utilizada. La siguiente tabla detalla las diferencias conceptuales entre ambas:

Característica	Árbol B	Árbol B+
Ubicación de los datos	Las claves y sus respectivos datos asociados pueden residir en cualquier nodo del árbol (tanto internos como hojas).	Los datos asociados se almacenan exclusivamente en los nodos hoja. Los nodos internos únicamente contienen claves guía para direccionar la búsqueda.
Enlace entre hojas	Los nodos hoja se encuentran aislados unos de otros.	Todos los nodos hoja se encuentran interconectados de forma secuencial mediante una lista enlazada.
Búsqueda por rangos	Compleja y lenta, debido a que requiere realizar recorridos alternantes (ascendentes y descendentes) en la estructura.	Sumamente eficiente, puesto que solo se localiza la primera coincidencia y luego se recorre linealmente la lista enlazada de las hojas.

- **Indexación de Bases de Datos**

El diseño de un sistema de archivos o base de datos busca evitar a toda costa el escaneo completo de tablas (*Full Table Scan*), un proceso ineficiente que obliga al hardware a leer cada sector físico del disco de manera secuencial.

El Concepto de Índice

Un índice en informática actúa de manera homóloga al índice analítico de un texto impreso. En lugar de examinar cada página para localizar un término específico, el sistema consulta una estructura organizada que

asocia cada clave de búsqueda con una dirección física o puntero de memoria.

Mecanismo de Búsqueda mediante Árbol B+

Los sistemas operativos y las bases de datos gestionan la información en bloques físicos de memoria (usualmente de 4 KB). Los nodos de un Árbol B+ se diseñan para coincidir exactamente con el tamaño de estos bloques, logrando que cada nodo consultado equivalga a una única lectura de disco.

El proceso de recuperación de un registro mediante su identificador (ID) se realiza de la siguiente manera:

1. **Acceso al Nodo Raíz:** El motor de la base de datos lee el bloque raíz, el cual provee los rangos de claves para determinar hacia cuál de los nodos hijos se debe realizar la transición.
2. **Navegación por Nodos Internos:** Se accede al nodo intermedio correspondiente, repitiendo el filtrado de rangos hasta aproximarse a la ubicación final.
3. **Localización en el Nodo Hoja:** Se llega al nodo hoja, el cual contiene la dirección física del registro almacenado en el disco.

Este método de ramificación múltiple permite que, incluso en conjuntos de datos que superan los miles de millones de registros, la búsqueda se complete con un número muy reducido de accesos a disco (típicamente entre 3 y 4 lecturas), reduciendo el tiempo de procesamiento a milisegundos.

- **Aplicaciones Prácticas**

La aplicabilidad de esta teoría se evidencia en los componentes críticos del software moderno:

- **Sistemas de Gestión de Bases de Datos Relacionales (RDBMS):**

- **MySQL (Motor InnoDB):** Emplea de manera predeterminada estructuras de Árbol B+ para la creación de índices agrupados en sus tablas, optimizando las consultas de claves primarias.
- **PostgreSQL:** Implementa de forma nativa la indexación basada en Árboles B, facilitando operaciones de ordenamiento, comparaciones de igualdad y consultas de rangos de alta velocidad.

- **Sistemas de Archivos de Sistemas Operativos:**

- **NTFS (Windows) y ext4 (Linux):** Utilizan variantes de Árboles B para organizar los directorios y archivos dentro de los discos de almacenamiento. Al buscar un archivo por su nombre, el sistema operativo emplea estas estructuras para ubicar de forma inmediata el sector físico del disco o unidad de estado sólido (SSD) donde se encuentra almacenado el archivo.

Los Árboles de Decisión en la Inteligencia Artificial: De la Teoría de Grafos al Aprendizaje Conectado

Introducción

En el ámbito de la Inteligencia Artificial (IA), uno de los mayores desafíos es lograr que las máquinas no solo procesen datos, sino que tomen decisiones lógicas de forma comprensible para los seres humanos. La respuesta más intuitiva a este problema proviene de la teoría de grafos: los árboles de decisión. Estos modelos matemáticos actúan como mapas de flujo que transforman grandes volúmenes de información en una secuencia ordenada de preguntas y respuestas. A diferencia de otros

modelos de aprendizaje automático modernos que funcionan como "cajas negras" indescifrables, los árboles de decisión destacan por su transparencia, permitiendo rastrear el camino exacto que siguió una computadora para llegar a una conclusión específica.

Estructura Fundacional: El Flujo de la Lógica

La estructura de un árbol de decisión imita la forma en que los humanos resolvemos problemas complejos, dividiéndolos en elecciones consecutivas de arriba hacia abajo. En términos técnicos, esta estructura se compone de tres elementos esenciales:

Nodo Raíz: Representa la pregunta inicial y el punto de partida del modelo. Evalúa el atributo más crítico del conjunto de datos.

Nodos Internos: Son las bifurcaciones o preguntas secundarias. Surgen a partir de las respuestas previas y dividen los datos en caminos cada vez más específicos.

Hojas: Constituyen los puntos finales del árbol. Aquí no se realizan más preguntas, ya que representan la predicción definitiva o el resultado final del proceso.

Este diseño permite que cualquier problema de clasificación o predicción se reduzca a un viaje lógico y lineal a través de ramas bien definidas.

Algoritmos de Construcción: El Arte de Saber Preguntar

Para que un árbol de decisión sea eficiente, la computadora debe aprender a formular las preguntas correctas en el orden adecuado. Esto se logra mediante el entrenamiento del modelo, guiado por métricas matemáticas que determinan "dónde cortar" o dividir los datos:

Impureza de Gini: Es una métrica que mide el nivel de desorden o mezcla en un grupo de datos. Un valor cercano a cero significa que los datos están perfectamente organizados y clasificados. El algoritmo busca constantemente las divisiones que dejen los datos lo más limpios posible.

Ganancia de Información: Calcula la cantidad de claridad que se obtiene tras realizar una pregunta. El sistema prioriza los atributos que reducen drásticamente la incertidumbre inicial.

El proceso de ramificación se detiene bajo criterios de corte específicos (como alcanzar una hoja completamente pura o un límite máximo de preguntas configurado). Esto evita que el árbol crezca en exceso y se vuelva tan específico que falle al enfrentarse a datos nuevos que nunca antes ha visto.

Evolución: El Poder Colectivo de los Bosques Aleatorios

A pesar de su claridad, un árbol de decisión individual presenta una debilidad: es altamente sensible a las variaciones en los datos. Un pequeño cambio en la información de entrada puede alterar por completo toda la estructura del árbol. Para superar esta limitación, la Inteligencia Artificial evolucionó hacia el concepto de Bosques Aleatorios (Random Forest).

Esta técnica de aprendizaje combina cientos de árboles de decisión independientes que trabajan en paralelo. Cada árbol analiza una porción distinta de los datos y genera su propia predicción. Finalmente, el sistema aplica un mecanismo de votación democrática donde el resultado definitivo es aquel que obtuvo la mayoría de votos entre todos los árboles del bosque. Esta evolución incrementa drásticamente la precisión y la estabilidad del modelo, convirtiéndolo en una de las herramientas más robustas del aprendizaje automático.

Aplicaciones Prácticas en el Mundo Real

La capacidad de estos modelos para justificar cada una de sus decisiones los hace indispensables en sectores estratégicos donde el error humano debe minimizarse y la transparencia es obligatoria:

Sector Financiero: Los bancos emplean árboles de decisión para automatizar la aprobación de créditos. El sistema evalúa variables como el nivel de ingresos y el historial de pagos para determinar el nivel de riesgo antes de conceder un préstamo.

Medicina y Salud: En los sistemas de diagnóstico asistido por computadora, ayudan a los médicos a identificar enfermedades cruzando síntomas, edad y antecedentes clínicos del paciente en un diagrama de flujo predictivo.

Comercio Electrónico: Funcionan como motores de recomendación sencillos pero eficaces, sugiriendo productos, películas o música según el historial de clics y las elecciones previas de los usuarios.

Conclusión

Los árboles de decisión representan un puente perfecto entre la lógica matemática pura y la practicidad del aprendizaje automático. Su evolución hacia los bosques aleatorios demuestra que la cooperación entre modelos simples puede resolver problemas de alta complejidad. Al ofrecer un equilibrio único entre precisión técnica y facilidad de comprensión, estos modelos siguen siendo una pieza fundamental en el desarrollo de una Inteligencia Artificial ética, auditable y accesible para la sociedad.

Fase 4: Proyecto Integrador - Modelado y Optimización de Redes

1. **Los equipos elaboran un informe técnico del proyecto que incluya: descripción del problema, modelado matemático (lógica, booleana, grafos), implementación en software, resultados obtenidos, análisis de optimización y conclusiones. (ARTICULO)**

INFORME TÉCNICO: Teoría de grafos para optimizar la red de cobertura a instituciones educativas públicas de la Provincia Sur del Sumapaz (Colombia)

1. Descripción del Problema

- **Contexto y Relevancia:** La Universidad de Cundinamarca, sede Fusagasugá, desarrolla proyectos de Ciencia, Tecnología e Innovación (CTI) e interacción social universitaria (ISU), como talleres de robótica educativa y pensamiento computacional. Estos programas buscan atender las necesidades de instituciones educativas públicas en la Provincia del Sumapaz; sin embargo, se ha identificado que la cobertura actual se limita a los municipios cercanos, excluyendo a poblaciones de zonas rurales y apartadas [4].
- **El Desafío Técnico:** La región presenta una geografía montañosa compleja y una infraestructura vial precaria, caracterizada por carreteras rurales no pavimentadas y en mal estado. Estas condiciones generan un desafío logístico significativo, traduciéndose en altos costos y un uso considerable de tiempo para el traslado de recurso humano y tecnológico [4].
- **Objetivos:** El estudio se propone determinar las rutas de viaje más rápidas desde la Universidad de Cundinamarca (sede Fusagasugá) hacia las instituciones educativas públicas más apartadas de la Provincia Sur del Sumapaz para optimizar la planificación de actividades y la asignación de recursos [4].

2. Modelado Matemático y Teórico

- **Lógica / Álgebra Booleana:** No especificado en el artículo original.

- **Teoría de Grafos:** El problema se modela mediante un grafo simple y ponderado definido como $G=(V,E)$ $G=(V,E)$ $G=(V,E)$ [4].
- **Vértices (V):** Representan 13 ubicaciones, incluyendo la sede universitaria (nodo origen), municipios, centros poblados y veredas [4].
- **Aristas (E):** Representan las conexiones viales entre los nodos (16 aristas en total) [4].
- **Pesos (w):** El peso de cada arista $w(u,v)$ $w(u,v)$ $w(u,v)$ corresponde al tiempo de viaje expresado en minutos [4].

3. Implementación en Software

- **Arquitectura:** La solución está estructurada mediante la recolección de datos de tiempos de viaje reales y su posterior procesamiento mediante un modelo de grafos para la ejecución de algoritmos de optimización [4].
- **Algoritmos y Lógica:** Se implementa el algoritmo de Dijkstra, el cual opera en tres fases principales:
 1. Marcación del nodo origen con tiempo cero.
 2. Exploración de vértices adyacentes y cálculo o actualización de tiempos acumulados.
 3. Selección del vértice no visitado con el peso mínimo para la siguiente iteración [4].
- **Tecnologías:**
 - Google Maps: Utilizado para la obtención de los tiempos de viaje entre nodos [4].
 - Python: Lenguaje empleado para desarrollar la aplicación que ejecuta el algoritmo de Dijkstra [4].

4. Resultados Obtenidos

- **Pruebas realizadas:** Se analizaron cuatro recorridos estratégicos desde el nodo origen (FU) hacia los destinos más alejados, evaluando diferentes alternativas de conexión para asegurar la ruta mínima [4].
- **Métricas y Datos:** Los resultados obtenidos fueron los siguientes:

Recorrido	Origen	Ruta Óptima	Destino Final	Tiempo Total (min)
1	FU	FU → SI → TI → CU	Cumaca	59
2	FU	FU → EB → BA	Bateas	68
3	FU	FU → EB → PA → VE → CA	Cabrera	146
4	FU	FU → AR → SB → SH → AN	Vereda Andes	150

5. Análisis de Optimización

Mejoras de Rendimiento: La aplicación del algoritmo permitió descartar rutas intuitivas menos eficientes. Por ejemplo, para llegar a Bateas se comparó la ruta vía Cumaca (100–110 minutos) con la ruta vía El Boquerón, obteniendo una reducción del tiempo a 68 minutos. De igual manera, para el tercer recorrido se obtuvo un tiempo de 146 minutos frente a otras alternativas de 177 y 232 minutos [4].

6. Conclusiones

- **Conclusiones del Autor:** Se demuestra que la teoría de grafos y el algoritmo de Dijkstra constituyen métodos eficaces y confiables para optimizar rutas en contextos rurales complejos. Además, el modelo proporciona una base cuantitativa para la logística y planificación de proyectos universitarios en zonas de difícil acceso [4].

- **Limitaciones y Trabajo Futuro:** Los tiempos calculados no consideran la duración de las visitas técnicas realizadas en cada institución. El artículo no especifica otras líneas de trabajo futuro [4].

Fase 5: Exposición, Defensa y Evaluación Final

1. **Reflexión final: los estudiantes elaboran una autoevaluación individual sobre su proceso de aprendizaje durante las tres unidades, identificando fortalezas, debilidades y metas de mejora para el siguiente ciclo académico(individual).**

Wilson Joel Gevara Moncada

La verdad es que entrar a la universidad a los 18 años y toparme con matemáticas discretas fue un choque muy fuerte, sobre todo porque en el colegio nunca vi bases de lógica y el inicio del semestre me costó bastante por el nivel de abstracción. Aunque a veces el tiempo se me vino encima y me tocó resolver bajo presión, la responsabilidad pudo más y siempre busqué la forma de sacar adelante la materia, especialmente en la unidad 2, donde los mapas de Karnaugh y las simplificaciones me exigieron un esfuerzo extra por su nivel de dificultad. Sin embargo, todo el panorama cambió en la unidad 3; el tema de árboles y grafos me pareció sumamente interesante y visual, lo que me ayudó a conectar todas las piezas de la asignatura de una manera mucho más fluida. Como meta para el próximo ciclo me propongo organizar mejor mis tiempos de estudio desde el primer día para evitar acumular la carga de trabajo, pero me deja muy satisfecho ver cómo esa lógica que tanto me costó al principio ahora la entiendo y sé que es la base para desarrollar tecnologías como la Inteligencia Artificial.

Josselin Estafania Nero Fernandez

La comprensión de los árboles y los grafos resulta fundamental nuestra carrera, ya que estas estructuras de datos trascienden la teoría matemática para convertirse en el soporte lógico de la infraestructura digital moderna. Mientras que los árboles imponen un orden jerárquico y estructurado que optimiza la búsqueda y organización sistemática de la información, los grafos ofrecen una versatilidad ilimitada para modelar redes complejas y dinámicas, tales como sistemas de transporte, telecomunicaciones, redes sociales y algoritmos de optimización de rutas. Así, el dominio de estas herramientas no solo es indispensable para el diseño de algoritmos eficientes y escalables, sino que también proporciona un marco conceptual crucial para traducir problemas caóticos del mundo real en soluciones de software lógicas, estructuradas y altamente funcionales.

Yuleisy Cecibel Jaramillo Jaramillo

En la primera Unidad constituyó la base absoluta para el desarrollo de mi pensamiento lógico. Como mencioné desde mi primer examen, me sorprendió profundamente ver cómo las matemáticas están presentes en todos lados, incluso cuando lógicamente hay que plantear y resolver problemas basados en la toma de decisiones. Sin embargo, reconozco que fue una unidad bastante complicada. Me tocó adaptarme a un nuevo entorno de aprendizaje universitario en el que ya no solo se trataba de aplicar fórmulas mecánicamente, sino de crear nuestras propias soluciones siguiendo una lista de leyes lógicas que, al principio, se complicó mucho memorizar y asimilar. A pesar de la dificultad del cambio de ritmo, considero que supe adaptarme con éxito basándome puramente en el esfuerzo, la constancia y el estudio dedicado.



La segunda unidad me permitió ver la utilidad práctica de la lógica matemática en el hardware y el software. Utilizar mapas de Karnaugh de hasta 4 variables fue una de las partes más didácticas del ciclo. Aprender a agrupar unos de manera visual para minimizar expresiones complejas me demostró el valor de la eficiencia y la optimización de recursos, destrezas indispensables para el diseño de futuros circuitos lógicos o la simplificación de condicionales en programación.

Y por último, la unidad 3 se profundizó sobre la estructuración de la información y las rutas de búsqueda mediante grafos, además de jerarquía de la misma mediante los grafos. Todo esto dio una mejor perspectiva de la verdadera aplicación de la matemática en software y bases de datos.

En conclusión, considero que mis fortalezas han sido la perseverancia y el deseo de aprender y en lo que se trata de temas he comprendido mejor lo que es mapas de Karnaugh y grafos, siendo mis debilidades la falta de organización en el tiempo. Para mejorar tendré que reforzar mis conocimientos técnicos como mi organización.

Sdier Emanuel Roblez Roblez

Durante las tres unidades fortalecí mis conocimientos sobre programación y estructuras de datos, comprendiendo mejor cómo aplicar la teoría para resolver problemas de manera lógica y organizada. Las actividades realizadas me ayudaron a desarrollar habilidades para analizar, interpretar y mejorar programas.

Considero que mi principal fortaleza fue la constancia para aprender y cumplir con las tareas propuestas. Sin embargo, también identifiqué que debo seguir reforzando algunos temas más complejos y practicar con mayor frecuencia para resolver ejercicios de forma más rápida y segura.

Como meta para el siguiente ciclo académico, me propongo continuar mejorando mi lógica de programación, profundizar en el estudio de estructuras de datos y dedicar más tiempo a la práctica para fortalecer mis conocimientos y afrontar nuevos retos con mayor confianza.

En general, este proceso fue muy valioso para mi formación, ya que me permitió avanzar tanto en mis conocimientos técnicos como en mi capacidad de aprendizaje y análisis.

6. Preguntas de Control

1. Explique la diferencia entre los algoritmos BFS y DFS. ¿En qué situaciones sería más apropiado utilizar BFS y en cuáles DFS? Argumente con ejemplos de aplicaciones reales en la ciencia de la computación.

Diferencias Fundamentales y Aplicaciones

Tanto BFS como DFS son algoritmos fundamentales en la teoría de los grafos utilizados para recorrer o buscar elementos dentro de una estructura de datos, ya sea árboles o grafos, sin embargo, tienen grades diferencias:

Características	BFS	DFS
Estrategia	Explora nivel por nivel. Primero visita todos los vecinos directos antes de pasar al siguiente nivel.	Explora a lo profundo. Sigue un camino hacia abajo hasta el límite antes de retroceder (<i>backtracking</i>).
Estructura de Datos	Estructura de datos en Cola: Rige su comportamiento bajo el principio FIFO (siglas en inglés para First In, First Out), que se traduce como: "El primero en entrar es el primero en salir".	Estructura de Datos en Pila: Una Pila es una estructura de datos que opera bajo el principio LIFO (siglas en inglés para Last In, First Out), que significa: "El último en entrar es el primero en salir".
Consumo de Memoria	Puede requerir mucha memoria en grafos muy anchos (almacena todos los nodos de un nivel).	Suele consumir menos memoria, limitada a la profundidad máxima del camino actual.

¿Cuándo utilizar BFS? (Casos y Ejemplos Reales)

El algoritmo BFS es ideal cuando el objetivo principal es encontrar el camino más corto o cuando se sabe que la solución se encuentra cerca del nodo de origen.

- **Situación óptima:** Búsqueda en grafos no ponderados donde se requiere el menor número de saltos o pasos posibles.
- **Ejemplos en la vida real:**
 - **Redes Sociales (Sugerencias de Amistad):** Plataformas como LinkedIn o Facebook emplean BFS para calcular los "grados de separación" entre usuarios. Permite identificar de manera eficiente quiénes son los amigos de los amigos (vecinos a dos niveles de distancia) para sugerir conexiones.

¿Cuándo utilizar DFS? (Casos y Ejemplos Reales)

El algoritmo DFS es preferible cuando se necesita **explorar todas las posibilidades** de manera exhaustiva, o cuando se requiere analizar la estructura de conectividad del grafo completo.

- **Situación óptima:** Escenarios de toma de decisiones jerárquicas, detección de ciclos o cuando la solución se encuentra en la parte más profunda del árbol.
- **Ejemplos en la vida real:**

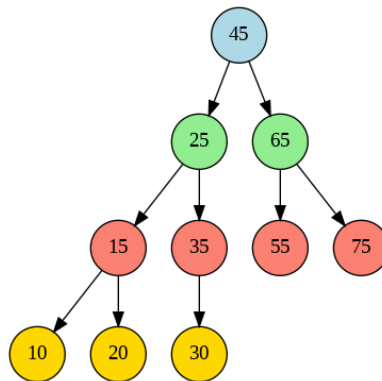
- **Inteligencia Artificial en Juegos (Ajedrez o Laberintos):** Los motores de juego exploran las ramificaciones de posibles jugadas futuras mediante DFS (o variantes como Minimax). El algoritmo simula una jugada hasta el final de la partida para evaluar si es ganadora antes de retroceder e intentar otra alternativa.

2. Construya un árbol binario de búsqueda (BST) insertando los siguientes valores en orden: 45, 25, 65, 15, 35, 55, 75, 10, 20, 30. Muestre el árbol resultante y realice los recorridos inorden, preorden y posorden. ¿Qué utilidad tiene cada recorrido en la programación?

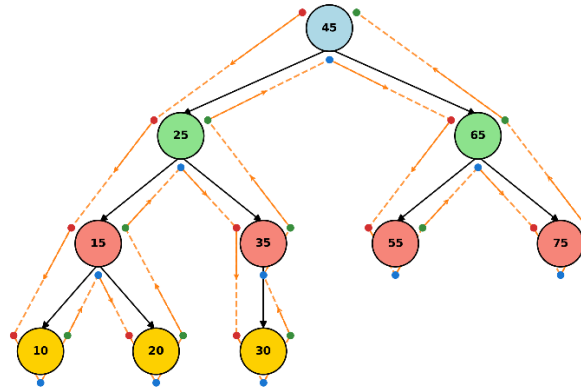
CONSTRUCCIÓN DEL ÁRBOL

- **Paso 1 (Insertar 45):** Al ser el primer elemento, se convierte en la raíz del árbol.
- **Paso 2 (Insertar 25):** Se compara con 45. Al ser menor ($25 < 45$), se coloca como su hijo izquierdo.
- **Paso 3 (Insertar 65):** Se compara con 45. Al ser mayor ($65 > 45$), se coloca como su hijo derecho.
- **Paso 4 (Insertar 15):** Se compara con 45 (menor, va a la izquierda) y luego con 25. Al ser menor ($15 < 25$), se inserta a la izquierda de 25.
- **Paso 5 (Insertar 35):** Se compara con 45 (menor, izquierda) y con 25. Al ser mayor ($35 > 25$), se coloca a la derecha de 25.
- **Paso 6 (Insertar 55):** Se compara con 45 (mayor, derecha) y con 65. Al ser menor ($55 < 65$), se coloca a la izquierda de 65.
- **Paso 7 (Insertar 75):** Se compara con 45 (mayor, derecha) y con 65. Al ser mayor ($75 > 65$), se coloca a la derecha de 65.
- **Paso 8 (Insertar 10):** Se compara con 45 (izq.), 25 (izq.) y 15. Al ser menor ($10 < 15$), se coloca a la izquierda de 15.
- **Paso 9 (Insertar 20):** Se compara con 45 (izq.), 25 (izq.) y 15. Al ser mayor ($20 > 15$), se coloca a la derecha de 15.
- **Paso 10 (Insertar 30):** Se compara con 45 (izq.), 25 (der.) y 35. Al ser menor ($30 < 35$), se coloca a la izquierda de 35.

Grafo final:



Puntos guía para la construcción de los recorridos



Siendo los puntos de color rojo los que nos ayuden a formar o definir el preorden, los puntos de color azul lo que es el inorden y los verdes para el postorden.

De esta manera si seguimos el recorrido que forman las flechas y en cada uno solo tomamos en cuenta los puntos del color del recorrido analizado. Teniendo lo siguiente:

Preorden:

$$PRE = 45, 25, 15, 10, 20, 35, 30, 65, 55, 75$$

Inorden:

$$IN = 10, 15, 20, 25, 30, 35, 45, 55, 65, 75$$

Postorden:

$$POST = 10, 20, 15, 30, 35, 25, 55, 75, 65, 45$$

Utilidad de recorrido:

Preorden:

En programación, el recorrido preorden se utiliza cuando se requiere procesar el nodo de origen antes de conocer o crear sus ramificaciones.

Aplicaciones reales en ingeniería de software:

- **Serialización y Persistencia de Objetos:** Para guardar la estructura de un árbol en un archivo de texto (como JSON o XML) o para transmitirla a través de una API de red, se utiliza preorden. Al reconstruir el árbol en memoria en el sistema receptor, leer el archivo en preorden garantiza que el nodo raíz e intermedios se creen antes de intentar asignarles nodos hijos.
- **Compiladores y Árboles de Sintaxis Abstracta (AST):** Cuando un compilador analiza código fuente (como $x = a + b$), genera un árbol jerárquico. El recorrido preorden se utiliza para la generación de código intermedio o para traducir el código a lenguajes basados en instrucciones de máquina prefijas.
- **Clonación de Estructuras Complejas (Deep Copy):** Copiar un objeto anidado en memoria requiere instanciar primero el objeto contenedor (raíz) y luego instanciar recursivamente sus atributos y dependencias (hijos).

Inorden:

En programación, el recorrido inorden se asocia de forma directa con el ordenamiento y la búsqueda eficiente.

Aplicaciones reales en ingeniería de software:

- Implementación de Diccionarios y Mapas Ordenados: Lenguajes de programación como C++ y Java estructuran sus colecciones internamente como árboles auto-balanceados. Para exportar todos los elementos de estas colecciones en orden alfabético o numérico, el compilador ejecuta internamente un recorrido inorden.
- Búsqueda Binaria sobre Estructuras Dinámicas: Permite transformar una estructura jerárquica en una lista ordenada en tiempo lineal $O(n)$, facilitando búsquedas rápidas posteriores o la aplicación de algoritmos que requieren datos previamente ordenados.

Postorden:

En programación, el recorrido postorden implementa una lógica de procesamiento de abajo hacia arriba (*Bottom-Up*). El programa no puede tomar una decisión sobre el nodo actual hasta haber resuelto todos sus subcomponentes.

Aplicaciones reales en ingeniería de software:

- **Liberación Dinámica de Memoria (Destructor):** En lenguajes de bajo nivel como C o C++, borrar un nodo intermedio sin haber borrado sus hijos provoca "fugas de memoria". El destructor de un objeto tipo árbol debe recorrerse en postorden para liberar primero la memoria de las hojas del nivel inferior e ir subiendo progresivamente hasta destruir la raíz de forma segura.
- **Motores de Hojas de Cálculo (Excel) y Resolución de Dependencias:** Para calcular el valor de una celda que depende de otras (por ejemplo, $C1 = A1 + B1$), el software de la hoja de cálculo modela las dependencias como un árbol. Mediante un recorrido postorden, se calculan primero las celdas independientes (las hojas) antes de intentar resolver la celda final (la raíz).
- **Sistemas de Archivos (Cálculo de Tamaños):** Para que un sistema operativo determine el peso total de un directorio, se ejecuta un algoritmo postorden. El sistema calcula primero el peso de los archivos individuales dentro de cada subcarpeta para finalmente sumar dichos valores y asignárselos al directorio raíz.

3. Analice cómo la teoría de grafos puede contribuir a reducir la brecha digital en Ecuador. Proponga un modelo de red de conectividad para una comunidad rural de la provincia de Loja, utilizando los conceptos y algoritmos aprendidos en esta unidad.

La brecha digital que existe en Ecuador, es la desigualdad de acceso, calidad y uso de tecnología, muchas veces ocasionado por la falta de recursos (ocasionado por el desperdicio en la instalación de la fibra óptica y la falta de estrategia en la colocación de antenas).

Es por esta razón los grafos son una solución para plasmar las conexiones entre diferentes lugares, encontrar las rutas más óptimas que permitan reducir material y encontrar los lugares estratégicos para poner antenas con mayor potencia.

Modelo de red de conectividad para el Canton Paltas

Implementaremos un grafo ponderado no dirigido (ponderado: valores en las aristas).

Definición de los Nodos (Vértices)

Para mantenerlo limpio, oficial y de gran impacto, nos enfocaremos en la cabecera cantonal y las parroquias oficiales de Paltas (que geográficamente se conectan entre sí formando un anillo y varias ramificaciones):

- N_1 : **Catacocha** (Centro urbano / Nodo central de distribución)
- N_2 : **Lourdes** (Parroquia urbana, pegada a Catacocha)
- N_3 : **Yamana** (Parroquia rural, hacia el oeste)
- N_4 : **Casanga** (Parroquia rural, hacia el suroeste, zona baja)
- N_5 : **Guachanamá** (Parroquia rural, zona alta/noroeste)
- N_6 : **Lauro Guerrero** (Parroquia rural, norte)
- N_7 : **Cangonamá** (Parroquia rural, norte-centro)
- N_8 : **San Antonio** (Parroquia rural, noreste)

El Grafo de Carreteras Completo (Conexiones y Distancias en Km)

A diferencia de un árbol, aquí sí existen **caminos alternativos (ciclos)**. Por ejemplo, puedes ir de Catacocha a Guachanamá directamente, o dar la vuelta por Cangonamá, o ir por el lado de Yamana. Esto es lo que hace rico a tu grafo.

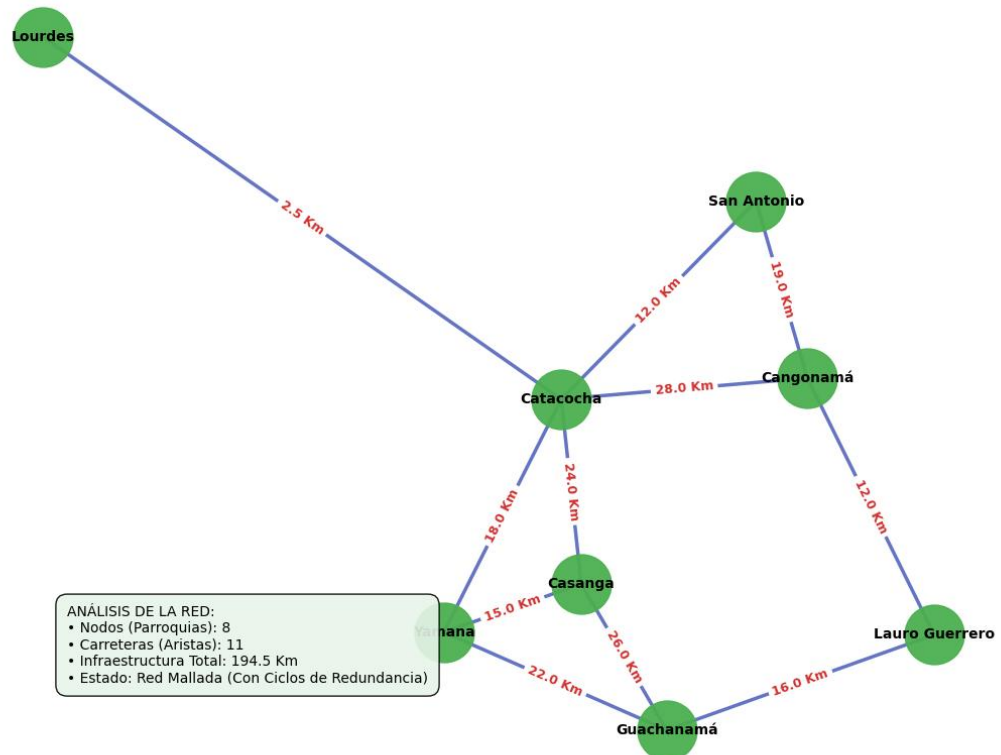
Aquí tienes la lista de aristas (E) con sus **valores reales/estimados en kilómetros**:

Conexiones

- **(Catacocha ↔ Lourdes):** 2.5 Km (Conexión urbana inmediata)
- **(Catacocha ↔ Yamana):** 18.0 Km (Vía colectora hacia el oeste)
- **(Catacocha ↔ Casanga):** 24.0 Km (Descenso hacia el valle)
- **(Catacocha ↔ San Antonio):** 12.0 Km (Salida norte)
- **(Catacocha ↔ Cangonamá):** 28.0 Km (Vía directa por la sierra)
- **(Yamana ↔ Casanga):** 15.0 Km (Conexión agrícola baja)
- **(Yamana ↔ Guachanamá):** 22.0 Km (Subida desde el valle a la cordillera)
- **(Guachanamá ↔ Lauro Guerrero):** 16.0 Km (Eje montañoso norte)
- **(Lauro Guerrero ↔ Cangonamá):** 12.0 Km (Conexión interparroquial norte)
- **(Cangonamá ↔ San Antonio):** 19.0 Km (Cierre del circuito norte)
- **(Casanga ↔ Guachanamá):** 26.0 Km (Ruta alternativa de conexión)

Grafo:

Grafo de Conectividad Interparroquial - Cantón Paltas



Elección de punto de antena:

Grado:

Vértice	Parroquia	Grado
N_1	Catacocha	5
N_2	Lourdes	1
N_3	Yamana	3
N_4	Casanga	3
N_5	Guachanamá	3
N_6	Lauro Guerrero	2
N_7	Cangonamá	3
N_8	San Antonio	2

El lugar donde se tiene el grado más alto o el mayor número de conexiones (carreteras) es Catacocha por lo que la antena se pondrá ahí.

Definición de los recorridos más cortos:

Desde Catacocha como cabecera cantonal se define los recorridos más cortos.

Tabla de Control de Dijkstra

Activo	N_1	N_2	N_3	N_4	N_5	N_6	N_7	N_8
N_1	—	[2.5]	18.0	24.0	∞	∞	28.0	12.0
N_2	—	—	18.0	24.0	∞	∞	28.0	[12.0]
N_8	—	—	[18.0]	24.0	∞	∞	28.0	—
N_3	—	—	—	[24.0]	40.0	∞	28.0	—
N_4	—	—	—	—	40.0	∞	[28.0]	—
N_7	—	—	—	—	40.0	[40.0]	—	—

Recorridos finales:

Parroquia de Destino	Ruta Más Corta	Distancia Total
Lourdes (N_2)	$N_1 \rightarrow N_2$	2.5 Km
San Antonio (N_8)	$N_1 \rightarrow N_8$	12.0 Km
Yamana (N_3)	$N_1 \rightarrow N_3$	18.0 Km
Casanga (N_4)	$N_1 \rightarrow N_4$	24.0 Km
Cangonamá (N_7)	$N_1 \rightarrow N_7$	28.0 Km
Guachanamá (N_5)	$N_1 \rightarrow N_3 \rightarrow N_5$	40.0 Km
Lauro Guerrero (N_6)	$N_1 \rightarrow N_7 \rightarrow N_6$	40.0 Km

De esta manera definimos conexiones mucho más eficientes y con menos gastos en material usando grafos.

7. Conclusiones

Se concluye que:

- Al aplicar conceptos sobre definiciones, terminología y tipos de grafos de manera práctica, al permitido comprender mejor estas bases, lo cual impulsa a ver los grafos como algo presente en el día a día.
- Al aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), de una manera práctica mejora no solo la comprensión de los temas sino su aplicación en la computación y en otros campos.
- Al utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos mejora la comprensión y precisión de los grafos generados.

Bibliografía

- [1] Wikipedia contributors, "Binary heap," Wikipedia, The Free Encyclopedia. [En línea]. Disponible: https://en.wikipedia.org/wiki/Binary_heap
- [2] GeeksforGeeks, "Binary Heap," GeeksforGeeks. [En línea]. Disponible: <https://www.geeksforgeeks.org/dsa/binary-heap/>
- [3] "An Efficient Sorting Technique for Priority Queues in High-Speed Networks," in Proc. IEEE Conf., 2019. [En línea]. Disponible: <https://ieeexplore.ieee.org/document/8822135/>
- [4] E. P. Vásquez Gómez, J. E. Quevedo Buitrago, D. O. Méndez Pineda, A. E. Merchán Hernández y W. D. Gordillo Ochoa, "Teoría de grafos para optimizar la red de cobertura a instituciones educativas públicas de la Provincia Sur del Sumapaz (Colombia)," Revista de la Universidad del Zulia, vol. 17, no. 48, pp. 298-319, 2026. doi: 10.5281/zenodo.18210299.